DB

Databricks Certified Generative AI Associate

100-Question Practice Set — Set 3 (ADVANCED)

Expert difficulty • scenario-based MCQ + open Q&A • with answer explanations

Section (weight)	Qs
1 · Design Applications (14%)	14
2 · Data Preparation (14%)	14
3 · Application Development (30%)	30
4 · Assembling & Deploying Apps (22%)	22
5 · Governance, Security & Privacy (8%)	8
6 · Evaluation & Monitoring (12%)	12
TOTAL	100

Exam: 45 scored MCQs · 90 minutes · $200 · 6 weighted sections

Set 3 · hardest questions · edge cases & multi-constraint scenarios · no overlap with Sets 1–2

How To Use This Set

This is the **hardest** of the three practice sets. Questions lean on edge cases, multi-constraint scenarios, and the fine distinctions between near-identical options that separate a pass from a fail. Expect **NOT/EXCEPT/NEXT** traps, 'choose TWO' items, and stems where several answers look plausible but only one survives every constraint in the prompt.

Distribution still mirrors the 2026 exam weighting, so Application Development (30%) and Assembling & Deploying (22%) dominate. Each item is a **multiple-choice question** (A–D, marked answer) or an **open question** demanding a precise, reasoned explanation — with a rationale covering *why the answer is right and why the distractors fail*.

These probe the newer 2026 material hard: **Agent Bricks limits, MCP vs tool-calling, AI Gateway (the ai_query rate-limit caveat, inference vs usage tables, legacy-table deprecation), MLflow 3 tracing/evaluation, Prompt Registry aliases, CLEARS, custom scorers, and Lakehouse Monitoring profiles.**

Suggested approach: treat this as a diagnostic. Cover the answer, commit to a choice AND a one-line reason, then check both. If you can defend the rationale (not just pick the letter), you're at pass level. These 100 do not repeat Sets 1–2 — you now have 300 unique questions.

Section 1 — Design Applications (14%)

Q1. MCQ

A regulated insurer needs an agent that (a) reads a claim PDF, (b) checks policy limits from a governed table, (c) flags fraud signals, and (d) must NEVER expose another customer's data even under prompt injection. The team proposes a single ReAct agent with three tools. Which design objection is MOST valid?

A. ReAct can't call tools, so it won't work at all
B. Tenant isolation must be enforced at the data/retrieval layer (UC row filters), not by the agent's reasoning, regardless of pattern
C. Only multi-agent supervisor can read PDFs
D. ReAct is always slower than planning, so it's disqualified

✓ **Answer: B**

> The non-negotiable requirement is data isolation under adversarial input; that must be enforced at the governance/retrieval layer (UC row/column security, scoped functions), because any agent's prompt-level reasoning can be subverted by injection. A is false (ReAct uses tools); C is false (extraction isn't supervisor-only); D is an overgeneralization, not the core objection.

Q2. OPEN

You must choose between (i) zero-shot CoT, (ii) few-shot, and (iii) few-shot CoT for a multi-step numeric reasoning task that ALSO requires a strict output schema. Explain the optimal combination and the failure mode of each alone.

✎ **Model answer:**

> Use few-shot CoT with an explicit output schema: examples teach both the reasoning style AND the exact format, while the CoT trigger improves multi-step arithmetic. Zero-shot CoT alone reasons well but is unreliable on strict format (no exemplars). Few-shot alone enforces format but may under-reason on multi-step math. Few-shot without CoT risks the model pattern-matching the answer shape while skipping intermediate steps. Practically: put reasoning in a hidden/scratch field and the final schema-constrained answer in a separate field, then parse only the latter.

Q3. MCQ

A latency-critical classification feature (p95 < 80ms) over 200M docs/day must pick a model. A teammate argues for a small decoder LLM with a 1-shot prompt 'because it generalizes.' What is the strongest counter?

A. Decoder LLMs are more accurate, so accept the latency hit
B. A fine-tuned encoder-only classifier gives far lower per-call latency and cost at this scale, with comparable or better accuracy for fixed-label classification
C. Use a seq2seq model for speed
D. Use few-shot with 10 examples to fix latency

✓ **Answer: B**

> At 200M/day with a hard p95, an encoder-only classifier is the architecture: single forward pass, no autoregressive decoding, lowest latency/cost, and strong accuracy on fixed labels. Generative 1-shot prompting adds decoding latency and token cost per call. Seq2Seq is slower still; more few-shot examples increase prompt tokens and latency.

Q4. OPEN

Distinguish when a Planning agent will OUTPERFORM a Multi-Agent Supervisor and vice versa, using the notions of task decomposition vs specialization.

✎ **Model answer:**

> Planning wins when one capable agent must execute interdependent, sequential sub-steps where ordering and intermediate state matter but no distinct expertise/parallelism is needed — a single plan-then-execute loop is simpler and cheaper. Supervisor wins when subtasks require DIFFERENT specialized agents/tools or can run in PARALLEL (e.g., a Genie analytics agent + a doc Knowledge Assistant + a compliance checker), and a coordinator must route and merge. Choose by asking: is the challenge ordering within one competence (Planning) or coordinating heterogeneous/parallel competences (Supervisor)?

Q5. MCQ

Which requirement set most strongly justifies Custom-code agents (LangGraph) over an Agent Bricks Supervisor, despite the loss of speed-to-production?

A. Standard doc Q&A with citations
B. Cyclic graphs with conditional branching, custom state persistence, and bespoke human-in-the-loop checkpoints the managed builder can't express
C. Wanting built-in MLflow tracing
D. Wanting auto-generated benchmarks

✓ **Answer: B**

> Custom frameworks are justified when control-flow needs (cycles, conditional edges, custom state, bespoke HITL checkpoints) exceed what the managed builder can express. B is exactly that. A is the canonical Knowledge Assistant case; C and D are advantages you KEEP with Bricks, so they argue against going custom.

Q6. MCQ

A four-part prompt has all four parts, yet outputs still vary wildly between 'a sentence' and 'three paragraphs.' Auditing reveals the Output Format says 'be concise.' What is the precise defect?

A. Missing Instruction
B. Output Format is under-specified — 'concise' is not measurable; it needs an explicit length/structure constraint (e.g., 'exactly one sentence, ≤25 words')
C. Missing Context
D. Temperature is too low

✓ **Answer: B**

> All parts are present; the defect is a vague Output Format. Subjective words ('concise') don't bound length. The fix is a measurable constraint. It's not a missing Instruction/Context, and low temperature would reduce, not cause, variability.

Q7. OPEN

Explain why naively adding more tools to a single Tool-Use agent can DEGRADE accuracy, and name two mitigations.

✎ **Model answer:**

> More tools enlarge the action space and the prompt (all tool schemas/descriptions), increasing the chance of wrong tool selection, argument errors, and context bloat that crowds out the task — sometimes called tool confusion. Mitigations: (1) tool curation/retrieval — expose only relevant tools per query (dynamic tool selection) or group into sub-agents; (2) sharper tool descriptions + few-shot tool-use exemplars + input validation, and a max-step limit to bound erroneous loops.

Q8. MCQ

You're designing memory for a 3-hour support session. Requirements: remember the resolved ticket ID and product, but stay within a tight token budget. Which composite memory design is best?

A. Full-history injection every turn
B. Windowed buffer of last N turns ONLY
C. Windowed buffer for recent turns PLUS a fact store holding durable facts (ticket ID, product)
D. No memory; re-ask the user each turn

✓ **Answer: C**

> Combine a short window for conversational coherence with a fact store that persists durable entities (ticket ID, product) cheaply — durable facts survive even after they leave the window. Full-history blows the budget; window-only forgets early facts; no memory is poor UX.

Q9. MCQ

A team must summarize 10-K filings with STRICT length control and domain-tuned phrasing, on-prem, with reproducible outputs. Modern large decoder LLMs score higher on generic benchmarks. What is the most defensible choice?

A. A fine-tuned Seq2Seq summarizer (e.g., BART/Pegasus-style) for tight length control and domain tuning
B. A 175B+ general decoder via external API
C. An encoder-only model
D. A diffusion model

✓ **Answer: A**

> The binding constraints — strict length control, domain-specific fine-tuning, on-prem, reproducibility — favor a fine-tuned Seq2Seq summarizer historically optimized for length-controlled summarization. A huge external decoder conflicts with on-prem/reproducibility; encoder-only can't generate; diffusion is for images.

Q10. OPEN

A designer claims 'RAG and fine-tuning are alternatives; pick one.' Give a scenario where you'd deliberately use BOTH, and explain the division of labor.

✎ **Model answer:**

> Use both when you need current factual grounding AND consistent domain behavior/format. Example: a medical assistant fine-tuned to adopt clinical tone, structured SOAP-note output, and safe refusal behavior (style/behavior baked into weights), combined with RAG over the latest guideline documents (volatile facts injected at query time). Fine-tuning handles HOW it answers; RAG handles WHAT current evidence it answers from. They are complementary, not mutually exclusive.

Q11. MCQ

Which is the BEST reason a Knowledge Assistant Brick might be chosen over hand-built RAG even by an expert team?

A. It allows arbitrary cyclic agent graphs
B. It auto-generates task-aware benchmarks/judges and grounding-loop optimization, collapsing weeks of eval/tuning into a governed managed flow
C. It removes the need for any documents
D. It guarantees zero hallucination

✓ **Answer: B**

> The differentiator is automated, task-aware evaluation/benchmark generation plus a managed grounding/optimization loop and governance — the part teams usually under-invest in. A describes custom frameworks; C is nonsense; no system guarantees zero hallucination.

Q12. MCQ

A multi-agent supervisor system keeps timing out on a long, multi-stage research task. Which built-in capability directly addresses this WITHOUT splitting the project into separate apps?

A. Increasing temperature
B. Supervisor long-running task mode (task continuation across multiple request/response cycles)
C. Reducing the number of retrieved chunks
D. Switching every subagent to keyword search

✓ **Answer: B**

> Supervisor long-running task mode breaks complex tasks into multiple request/response cycles via task continuation to avoid timeouts — built for exactly this. Temperature, chunk count, and search type don't address orchestration timeouts.

Q13. OPEN

Explain the security distinction between (a) a Unity Catalog function exposed as an agent tool and (b) an MCP-connected external SaaS tool, in terms of where governance is enforced.

✎ **Model answer:**

> A UC function tool runs inside the governance boundary: permissions, lineage, and data masking are enforced by Unity Catalog on the underlying data/function, so the agent inherits least-privilege automatically. An MCP-connected external tool reaches outside the lakehouse; governance is enforced at the connection/runtime layer — Managed OAuth connectors handle credentials/identity, and AI Gateway applies identity, permissions, observability, and guardrails to the tool calls. Both are governed, but UC governs the data/function natively while AI Gateway+OAuth govern the external interaction.

Q14. MCQ

For an image-and-text input that must output a structured risk score AND a generated rationale, which architecture composition is correct?

A. Encoder-only text model alone
B. A multimodal (vision-language) model for understanding, with a constrained generation step + parser for the structured score and rationale
C. A pure embedding model
D. A seq2seq translation model

✓ **Answer: B**

> Mixed image+text understanding requires a multimodal model; the structured score + rationale need constrained generation and parsing. Encoder-only and pure embedding models can't generate; a translation seq2seq model doesn't fit vision input or risk scoring.

Section 2 — Data Preparation (14%)

Q15. MCQ

Embedding model max sequence length is 512 tokens. Your structure-aware chunker emits some 700-token sections (long tables). Retrieval quietly degrades on those sections. What is happening and the best fix?

A. Nothing is wrong; long chunks are fine
B. Over-length chunks are silently truncated to 512 tokens at embedding time, losing tail content; split long sections to fit (with overlap) or use a longer-context embedder
C. Increase temperature
D. Switch the generator model

✓ **Answer: B**

> Inputs beyond the embedder's max length are truncated, so the embedding represents only the first 512 tokens — tail content becomes unretrievable. Fix by sub-splitting long sections within the limit (preserving overlap) or adopting an embedding model with a larger context window. Temperature/generator choice are irrelevant to embedding truncation.

Q16. OPEN

You must support BOTH precise lookups of exact part numbers (e.g., 'GX-44A/2') AND semantic questions over the same catalog. Explain why dense-only retrieval underperforms and what to implement.

✎ **Model answer:**

> Dense embeddings blur rare, exact tokens like alphanumeric SKUs, so similarity search may miss or rank them poorly. Implement hybrid retrieval: combine dense semantic search with a lexical/keyword (e.g., BM25-style) component so exact strings are matched precisely, then fuse/rerank. Optionally store the part number as filterable metadata for exact-match filtering. This covers both exact and semantic needs.

Q17. MCQ

A Delta Sync vector index shows correct counts but returns embeddings inconsistent with current text after a model upgrade. Root cause?

A. The index was not re-embedded with the new model; sync propagates text/rows but you must re-embed when the embedding model changes
B. Change Data Feed is disabled
C. Chunk overlap too high
D. Temperature drift

✓ **Answer: A**

> Switching embedding models invalidates existing vectors — they live in a different space. A row-count-correct sync still holds stale vectors unless you re-embed the corpus with the new model and rebuild/refresh the index. CDF being off would affect row freshness, not model-version mismatch; overlap/temperature are unrelated.

Q18. OPEN

Define and contrast Context Precision and Context Recall, then explain which one a too-small chunk size tends to harm and why.

✎ **Model answer:**

> Context Precision = fraction of retrieved chunks that are actually relevant (signal-to-noise of retrieval). Context Recall = fraction of the relevant information present in the corpus that was successfully retrieved (coverage). Too-small chunks tend to harm Context Recall: a single idea gets fragmented across many tiny chunks, so a fixed top-k may fail to gather all the pieces needed to answer, missing relevant content. (They can also help precision per chunk, which is the trade-off.)

Q19. MCQ

For a RAG corpus that updates continuously AND requires GDPR right-to-erasure, which chunk-storage design is REQUIRED (not merely nice-to-have)?

A. Store chunks with stable source_id/chunk_id keys in a Delta table with CDF, synced to the vector index
B. Store all chunks in one giant blob
C. Randomize chunk IDs each ingestion
D. Keep chunks only in the vector index with no source mapping

✓ **Answer: A**

> Erasure requires targeted deletion: stable source/chunk IDs in a CDF-enabled Delta table let you delete exactly the affected rows, and the synced index removes the corresponding vectors. A blob, randomized IDs, or index-only storage with no mapping make precise deletion impossible/unreliable.

Q20. MCQ

Retrieval shows high recall but the generator still answers poorly because the most relevant chunk is ranked 9th of 10 and gets truncated by the prompt budget. Best single fix?

A. Increase k to 50
B. Add a reranker (cross-encoder) to promote the most relevant chunk into the top few passed to the LLM
C. Remove all metadata
D. Lower the generator temperature

✓ **Answer: B**

> The relevant chunk is retrieved but ranked too low to survive the prompt budget — a ranking problem. A reranker re-scores candidates so the best one enters the top slots actually sent to the LLM. Raising k worsens budget pressure; metadata removal and temperature don't fix ranking.

Q21. OPEN

Why might lowercasing and aggressive normalization HURT retrieval for some corpora? Give a concrete example.

✎ **Model answer:**

> Normalization that's too aggressive can destroy meaningful distinctions. Example: case and punctuation carry meaning in code, identifiers, chemical formulas, or names ('US' vs 'us'; 'CaCO3'; 'iOS'). Lowercasing/stripping can collapse distinct tokens, merge unrelated concepts, and degrade exact matches. Normalization should be corpus-aware: preserve case/symbols where they're semantically significant, normalize only genuine noise (encoding errors, boilerplate, inconsistent whitespace).

Q22. MCQ

A pipeline embeds 5M chunks nightly with an expensive large embedder; only ~2% of source rows change daily. The bill is unsustainable. Best optimization that preserves quality?

A. Embed everything nightly but at higher temperature
B. Incremental embedding: use CDF/Delta Sync to embed only changed/new rows, not the whole corpus
C. Switch to a tiny embedder for all rows
D. Stop indexing

✓ **Answer: B**

> With only 2% churn, full nightly re-embedding is wasteful. Incremental embedding via Change Data Feed / Delta Sync re-embeds only changed/new rows, cutting cost ~50x while preserving quality (same model). A is nonsensical (no temperature in embedding); a tiny embedder trades quality; stopping indexing breaks search.

Q23. OPEN

You ingest HTML, PDF, and scanned-image documents. Explain the distinct preprocessing risk each introduces for RAG and how you'd address scanned images specifically.

✎ **Model answer:**

> HTML risk: boilerplate (nav/ads/footers) pollutes embeddings — strip it. PDF risk: broken reading order/columns and embedded tables/images — use a layout-aware parser and preserve structure/metadata. Scanned-image risk: no text layer at all — text extraction returns nothing; you must run OCR to produce a text layer before chunking/embedding, and validate OCR quality (errors propagate into retrieval). All three should converge to clean, normalized text with consistent metadata before a single shared embedder.

Q24. MCQ

Two near-duplicate policy versions exist; users complain answers cite the outdated one. Which combined data-prep approach is best?

A. Index both with no metadata
B. Add effective-date metadata, filter/boost by recency at query time, and deduplicate superseded versions
C. Increase chunk overlap
D. Raise k

✓ **Answer: B**

> Recency correctness needs metadata (effective date) plus query-time filtering/boosting and removal/superseding of obsolete versions. Indexing both blindly causes the exact problem; overlap and higher k don't encode recency.

Q25. MCQ

Which statement about embedding distance metrics is correct in a production index?

A. You can query a cosine-built index with L2 and get identical rankings
B. The query must use the metric the index was built for; mixing metrics yields incorrect or meaningless rankings
C. Metric choice never affects results
D. Dot product and cosine are identical for non-normalized vectors

✓ **Answer: B**

> Rankings depend on the configured metric; you must query with the metric the index was built for. Cosine vs L2 vs dot can rank differently, especially for non-normalized vectors (dot ≠ cosine unless normalized). So C and D are false, and A is incorrect.

Q26. OPEN

Explain why storing the original (un-normalized) chunk text alongside the embedded/normalized text matters specifically for citation faithfulness.

✎ **Model answer:**

> You may normalize text to improve embedding consistency, but the generator should be grounded on, and cite, the ORIGINAL wording so quotes/citations match the source verbatim and remain auditable. If you fed only normalized text, citations could differ from the source document (case/punctuation/format changes), undermining trust and traceability. Keeping both lets you embed on normalized text but ground/cite on original text.

Q27. MCQ

A team sets chunk overlap to 60% to 'be safe.' What is the most likely consequence?

A. Better retrieval with no downside
B. Index bloat, higher storage/embedding cost, and more redundant near-duplicate hits that can crowd the top-k with repeated content
C. Lower latency
D. Smaller index

✓ **Answer: B**

> Excessive overlap massively inflates the number of highly similar chunks, raising storage/embedding cost and causing the top-k to fill with near-duplicate passages — reducing context diversity. Modest overlap (~10-20%) is typical; 60% is usually counterproductive.

Q28. OPEN

You must chunk source code for a code-RAG assistant. Why is fixed-token chunking especially harmful here, and what's the better strategy?

✎ **Model answer:**

> Fixed-token chunking can split functions/classes mid-body, separating signatures from logic and breaking syntactic units, so retrieved chunks are uncompilable fragments that confuse the model. Better: structure-aware (AST/symbol-based) chunking that keeps whole functions/classes (or logical blocks) together, attaches metadata (file path, symbol name, language), and uses overlap only at logical boundaries — preserving self-contained, meaningful units.

Section 3 — Application Development (30%)

Q29. MCQ

You migrate a RAG generator from GPT-4 (8k usable) to a self-hosted 4k-context model and now hit context-overflow errors. You MUST keep the model and the 8-chunk retrieval. Which TWO changes resolve it with least quality loss?

A. Decrease chunk size AND add a reranker so fewer, shorter, higher-quality chunks fit
B. Increase max_output_tokens AND raise k
C. Use a smaller embedding model AND increase overlap
D. Retrain with ALiBi AND lower temperature

✓ **Answer: A**

> Total prompt = system + retrieved context + query + output budget. With fixed k=8 and a 4k window, shrink each chunk and rerank so the 8 slots carry the most relevant, shorter content — reducing tokens while preserving answer-bearing context. B increases tokens; embedding size/overlap don't reduce prompt length; ALiBi requires retraining (excluded) and temperature doesn't change length.

Q30. OPEN

An OpenAI-SDK client pointed at a Databricks Foundation Model endpoint returns 401 intermittently in a deployed job but works in the notebook. Walk through the most likely causes and the correct fix.

✎ **Model answer:**

> Likely causes: the notebook uses an interactive/user token while the job runs under a different principal (service principal) lacking endpoint permissions; or the token is short-lived/expired in the long-running job; or base_url/token were hard-coded for the notebook context. Fix: authenticate the job's principal correctly — grant the service principal CAN QUERY on the endpoint, supply credentials via Databricks secret scopes (not hard-coded), and ensure base_url targets the serving endpoint. Use the platform's managed auth rather than a personal token so it doesn't expire mid-job.

Q31. MCQ

A tool-using agent intermittently calls 'refund(amount)' BEFORE 'lookup_policy_limit()', occasionally issuing over-limit refunds. The order is logically required. Best fix?

A. Tell the model in the prompt to 'please call tools in order' and hope
B. Enforce the dependency in a deterministic chain/workflow (or gate refund on a validated limit), not in model discretion
C. Raise temperature for creativity
D. Remove the policy tool

✓ **Answer: B**

> When ordering is a hard business invariant, encode it deterministically: a workflow that always fetches the limit first and a guard that rejects refunds exceeding it. Relying on prompt wording is non-deterministic and unsafe; temperature worsens it; removing the policy tool removes the safeguard.

Q32. MCQ

For batch scoring 50M rows with an LLM via SQL, which is the correct and most scalable construct, and what is its key governance caveat in 2026?

A. ai_query(); caveat: AI Gateway does NOT enforce rate limits on ai_query batch — control cost via permissions/system-table alerts
B. A Python for-loop calling the REST API row by row
C. collect() then map in pandas
D. ai_query(); caveat: it cannot be governed at all

✓ **Answer: A**

> ai_query() runs LLM inference inside SQL/Spark, scaling across the cluster. Key 2026 caveat: AI Gateway provides usage TRACKING but does NOT rate-limit ai_query batch workloads, so cost control relies on endpoint permissions and system-table monitoring/alerts. Row-by-row loops and collect()+pandas don't scale; D is false (it is governed, just not rate-limited).

Q33. OPEN

Explain how to make a RAG chain's output both schema-valid AND faithful, and why a single mechanism is insufficient.

✎ **Model answer:**

> Schema validity and faithfulness are orthogonal. Use a structured output parser (with retry/repair) to guarantee the JSON shape, AND a grounding instruction ('answer only from context; cite; say I don't know') plus a faithfulness scorer/guardrail to ensure the content is supported by retrieved context. A parser alone yields well-formed but possibly hallucinated content; a faithfulness check alone yields grounded but possibly malformed output. You need both: shape (parser) + substance (grounding + faithfulness).

Q34. MCQ

An agent loops: it keeps re-calling a failing search tool with the same query, never terminating. Besides a max-iteration cap, which design change reduces repeated identical failures MOST?

A. Feed the tool error back as an observation and require the next Thought to change strategy (reformulate query or switch tool)
B. Increase temperature so it tries random tools
C. Hide the error from the model
D. Remove the search tool entirely

✓ **Answer: A**

> ReAct's self-correction depends on the error being surfaced as an observation and the next reasoning step adapting (new query/tool). Hiding the error (C) removes the signal it needs; random tool choice (B) is unreliable; removing the tool (D) cripples capability. The max-iteration cap is the safety net, but adaptive error handling is the real fix.

Q35. MCQ

Two prompt versions must be compared fairly before promotion. Which protocol is methodologically correct?

A. Run each version on the SAME fixed evaluation dataset with the SAME scorers (built-in + custom), logging prompt_version per run
B. Test v2 on harder examples than v1 to stress it
C. Compare on whatever production traffic each happened to receive
D. Eyeball five outputs each

✓ **Answer: A**

> Fair comparison requires holding data and scorers constant across versions and tracking the version as a parameter. Different datasets (B), unmatched traffic (C), or eyeballing (D) confound the comparison. This is the registry-evaluate pattern.

Q36. OPEN

A streaming chat endpoint must also be evaluated and traced. Explain how token streaming, MLflow Tracing, and inference tables coexist for one deployed agent.

✎ **Model answer:**

> Streaming controls how tokens are returned to the client (incrementally) — a serving/transport concern. MLflow Tracing instruments the app to emit structured spans (retrieval, LLM call with token counts, tools) regardless of streaming. AI Gateway inference tables persist the request/response payloads and can store the MLflow traces for the agent. So the same request is streamed to the user, traced for observability, and logged to the inference table for monitoring/evaluation — three independent layers operating together.

Q37. MCQ

You need deterministic regression tests for a chain, but the model exposes only temperature and top_p (no seed). What is the most you can do, and the honest limitation?

A. Set temperature=0 (greedy); near-deterministic but minor nondeterminism can remain from batching/hardware/model updates
B. Set temperature=2 for stability
C. Determinism is guaranteed by lowering top_p only
D. Use few-shot to force identical outputs

✓ **Answer: A**

> temperature=0 (greedy decoding) maximizes determinism, but without a seed and given floating-point/batching effects and possible served-model updates, exact reproducibility isn't guaranteed — so assert on semantics/structure (scorers) rather than exact strings. High temperature increases variance; top_p alone doesn't ensure determinism; few-shot doesn't force identical text.

Q38. MCQ

A chain must call a governed UC function as a tool, but only for users authorized to see the underlying table. Where is authorization correctly enforced?

A. In the system prompt instruction
B. By Unity Catalog permissions on the function/table, evaluated under the calling principal's identity
C. By the LLM deciding who's allowed
D. By client-side JavaScript

✓ **Answer: B**

> Authorization must be enforced by UC under the actual calling identity, so unauthorized users physically cannot execute the function/read the data. Prompt instructions and LLM 'decisions' are bypassable; client-side checks are trivially circumvented.

Q39. OPEN

Explain the difference between logging a chain with the langchain flavor vs wrapping it as a PyFunc, and give one concrete case where PyFunc is REQUIRED.

✎ **Model answer:**

> The langchain flavor serializes a recognized LangChain object with minimal code and known serving semantics. PyFunc wraps arbitrary Python (custom predict, pre/post-processing, multiple model calls, non-LangChain orchestration) behind MLflow's standard interface. PyFunc is REQUIRED when the app's logic isn't a single serializable LangChain object — e.g., a custom router that calls different endpoints, applies bespoke business rules, and merges tool outputs with non-LangChain code. Then PyFunc gives you one deployable, signed artifact.

Q40. MCQ

A RAG app must answer 'what changed between policy v3 and v4?' Single-vector retrieval keeps returning chunks from only one version. What design addresses this best?

A. Increase temperature
B. Query decomposition/multi-query retrieval (retrieve for v3 and v4 separately, with version metadata filters) then synthesize a comparison
C. One huge chunk per document
D. Lower k to 1

✓ **Answer: B**

> Comparison questions need information from BOTH versions; a single similarity query biases toward one. Decompose into sub-queries (per version) with metadata filters, retrieve each, then have the LLM synthesize the diff. Temperature, giant chunks, and k=1 all worsen coverage.

Q41. OPEN

Why can a higher k sometimes DECREASE answer quality even when retrieval recall improves? Frame it in terms of the generator.

✎ **Model answer:**

> Higher k adds more context, raising recall, but also injects more irrelevant/low-relevance passages (noise) and consumes context-window tokens. The generator can be distracted by irrelevant content ('lost in the middle' effects), dilute attention on the key passage, or truncate the budget for reasoning/output. So beyond an optimal k, added noise and token pressure degrade faithfulness and relevance despite better raw retrieval. Tune k with reranking to balance recall against generator signal-to-noise.

Q42. MCQ

For an OpenAI-compatible Databricks endpoint, which configuration is necessary and sufficient to use the OpenAI Python SDK?

A. Set base_url to the Databricks serving endpoint and use a Databricks token as api_key; call the chat/completions interface
B. Convert the model to GGUF first
C. Only set the OPENAI_ORG variable
D. Nothing; it auto-detects Databricks

✓ **Answer: A**

> Point base_url at the Databricks endpoint and authenticate with a Databricks token as the API key; then use the standard chat/completions calls. No format conversion (GGUF) is needed; org variables are irrelevant; it does not auto-detect.

Q43. MCQ

A guardrail must block responses that leak PII AND off-topic financial advice. A teammate proposes raising temperature to 'add variety so it avoids canned leaks.' Evaluate.

A. Reasonable — variety reduces leaks
B. Wrong — temperature affects randomness, not policy; use input/output content filters, PII detection/masking, and topic classifiers as guardrails
C. Correct only for PII
D. Correct only for advice

✓ **Answer: B**

> Temperature changes sampling randomness, not whether content violates policy — and higher temperature can INCREASE unsafe variability. Guardrails are the right tool: PII detection/masking, content filters, and topic/scope classifiers on inputs and outputs.

Q44. OPEN

Describe how prompt aliases enable a zero-code-change rollback, and what must be true about the application code for this to work.

✎ **Model answer:**

> The app references a stable prompt alias (e.g., 'production') rather than a hard-coded prompt or a specific version number. Promotion repoints the alias to a new version; rollback repoints it to the last good version — no code change or redeploy needed. For this to work, the code must resolve the prompt by alias at runtime (fetch from the registry), not embed the prompt text or pin a fixed version, so changing the alias target immediately changes behavior.

Q45. MCQ

An agent's accuracy is fine but p99 latency is terrible because it always calls 3 tools sequentially even when 1 suffices. Best improvement?

A. Increase max output tokens
B. Make tool use conditional (the model/router calls tools only when needed) and parallelize independent tool calls
C. Add more tools
D. Raise temperature

✓ **Answer: B**

> Unconditional sequential tool calls inflate latency. Let the model/router invoke tools only when required and run independent calls in parallel to cut wall-clock time. More tools, more tokens, or higher temperature don't reduce unnecessary sequential calls.

Q46. MCQ

Which is the correct interpretation of a model 'signature' failing validation at serving time for a GenAI endpoint?

A. The GPU is overheating
B. The request's input schema doesn't match the declared signature (wrong/missing fields or types), so serving rejects it before inference
C. The embedding model is too small
D. The prompt is too creative

✓ **Answer: B**

> A signature defines expected input/output schema; a validation failure means the incoming request doesn't conform (missing/extra fields, wrong types). It's a contract mismatch caught pre-inference, unrelated to hardware, embeddings, or 'creativity'.

Q47. OPEN

Explain why setting max_output_tokens too high can cause failures even when the model 'should' be fine, in a long-context RAG prompt.

✎ **Model answer:**

> The context window bounds input + output together. A long retrieved-context prompt plus a very high max_output_tokens reservation can exceed the total window, causing a context-length error before or during generation — or forcing truncation of input. It also wastes cost/latency by allowing over-long generations. Set max_output_tokens to the realistic answer size so input + output fit the window with margin.

Q48. MCQ

A developer wants the SAME quality scorers used in development to also run on live production traffic. Which capability makes this possible without re-implementing metrics?

A. MLflow 3 evaluation/monitoring reuses the same built-in/custom scorers across dev and production (over traces/inference data)
B. Photon
C. Delta time travel
D. A second codebase for prod metrics

✓ **Answer: A**

> MLflow 3 unifies evaluation and monitoring so identical judges/scorers run in development and against production traces/inference data — consistency by design. Photon and time travel are unrelated; maintaining a separate prod metric codebase is exactly what this avoids.

Q49. MCQ

A chain intermittently returns valid JSON but with hallucinated field values not present in context. The parser passes it downstream. What is the correct layered fix?

A. Trust the parser; JSON validity implies correctness
B. Keep the parser for shape, ADD grounding constraints + a faithfulness/correctness scorer (and optionally citations) to catch unsupported values
C. Increase temperature
D. Remove the schema

✓ **Answer: B**

> Valid shape ≠ true content. Keep the parser for structure but add grounding instructions and a faithfulness/correctness check (and citations to source) so unsupported field values are detected/blocked. Trusting JSON validity (A) is the error; temperature and removing the schema don't address faithfulness.

Q50. OPEN

You must let an agent search the web for fresh facts but prevent it from acting on injected instructions inside retrieved web pages. Describe the mitigations.

✎ **Model answer:**

> Treat retrieved web content as untrusted DATA, not instructions: clearly delimit/quote it, instruct the model that retrieved text is reference-only and must not change its task, and never let tool outputs directly trigger privileged actions. Add guardrails for prompt-injection/data-exfiltration, restrict tool permissions (least privilege), require confirmation for side-effectful actions, and validate/strip suspicious instruction-like patterns. Keep system instructions separate and authoritative over any retrieved content.

Q51. MCQ

Which scenario REQUIRES provisioned throughput rather than pay-per-token, even though pay-per-token is cheaper at low volume?

A. Sporadic internal testing
B. A customer-facing feature with steady high QPS and a strict latency SLA needing guaranteed, predictable capacity
C. A nightly batch job
D. A one-off demo

✓ **Answer: B**

> Strict latency SLAs at steady high volume demand reserved, predictable capacity — provisioned throughput. Sporadic testing, one-off demos, and even batch jobs are better served by pay-per-token (or batch) since they don't need guaranteed real-time capacity.

Q52. MCQ

A RAG assistant must cite sources. Retrieval returns the right chunk, but citations point to the wrong page. Most likely pipeline defect?

A. Temperature too high
B. Chunk metadata (page/section) was lost or misaligned during extraction/chunking, so citations don't map to the true location
C. k too low
D. Embedding model too small

✓ **Answer: B**

> Wrong-page citations indicate the page/section metadata wasn't captured or got misaligned to the chunk text during extraction/chunking. Fix the ingestion to attach accurate positional metadata per chunk. Temperature, k, and embedding size don't determine citation mapping.

Q53. OPEN

Explain why 'just fine-tune the model on our docs' is often the wrong first response to hallucination in a doc-Q&A app, ordered against cheaper interventions.

✎ **Model answer:**

> Hallucination in doc-Q&A usually stems from missing grounding, not missing capability. Cheaper, faster first steps: (1) add grounding context + 'answer only from context / say I don't know'; (2) implement/improve RAG and retrieval quality (chunking, reranking, k); (3) add citations + a faithfulness check/guardrail. Fine-tuning is expensive, slow, and bakes facts into weights that go stale — it doesn't fix volatile-fact grounding and risks new errors. Reserve fine-tuning for style/format/behavior adaptation, not current-fact grounding.

Q54. MCQ

To reduce repeated cost on a high-traffic FAQ bot where 30% of queries are paraphrases of each other, which is the most effective and correct approach?

A. A semantic cache keyed on query embeddings (with TTL + invalidation on source change) to reuse answers for paraphrased queries
B. Exact-string cache only
C. Disable retrieval
D. Increase k to 50

✓ **Answer: A**

> Paraphrases won't match an exact-string cache. A semantic cache matches by embedding similarity, reusing answers across paraphrased queries, with TTL and invalidation to avoid staleness. Disabling retrieval or raising k don't reduce repeat cost and harm quality.

Q55. OPEN

A chain works in the notebook but fails on the serving endpoint with a missing-dependency error. Explain the root cause and the correct prevention.

✎ **Model answer:**

> The notebook environment had a library that wasn't captured in the logged model's environment, so the serving container — built from the model's recorded dependencies — lacks it. Prevention: log the model with accurate pip/conda requirements (and an input example so MLflow can infer/validate the environment and signature), pin versions, and test-load the model in a clean environment before deploying. Serving rebuilds from the model's recorded environment, not your notebook.

Q56. MCQ

Which is the BEST description of when to use a deterministic workflow vs an autonomous agent for a multi-step task?

A. Always use an autonomous agent; it's more advanced
B. Use a deterministic workflow when steps/ordering are known and fixed (reliability, auditability); use an agent when the path is dynamic and must be decided at runtime
C. Always use a workflow; agents are unsafe
D. They are interchangeable in all cases

✓ **Answer: B**

> Fixed, known step sequences favor deterministic workflows for reliability and auditability; genuinely dynamic tasks where the next step depends on intermediate results favor autonomous agents. Neither is universally superior, and they aren't interchangeable for arbitrary tasks.

Q57. MCQ

A RAG chain passes retrieved context as a single concatenated string with no separators or source tags. Faithfulness is fine but the model frequently attributes facts to the wrong document. Best fix?

A. Raise temperature
B. Structure the context with explicit per-chunk delimiters and source/ID tags so the model can attribute and cite each fact to its origin
C. Concatenate even more tightly to save tokens
D. Switch to keyword search

✓ **Answer: B**

> Mis-attribution despite good faithfulness means the model can't tell where one source ends and another begins. Delimiting each chunk and tagging it with its source/ID lets the model attribute and cite correctly. Temperature, tighter concatenation, and search-type changes don't fix attribution structure.

Q58. OPEN

A team reports their agent 'works great in the AI Playground but fails in production.' List the most common gaps between Playground validation and a deployed agent, and how to close them.

✎ **Model answer:**

> Common gaps: (1) Identity/permissions — Playground runs as the developer; production runs under a service principal that may lack endpoint/tool/data grants (fix: grant the prod principal least-privilege access). (2) Environment/dependencies — Playground has the dev libraries; serving rebuilds from the logged model's recorded deps (fix: capture accurate requirements + input example, test-load in a clean env). (3) Inputs — production traffic is messier/adversarial than hand-typed Playground prompts (fix: evaluate on a representative golden set incl. edge/adversarial cases; add guardrails). (4) Scale/latency — cold starts, rate limits, and concurrency don't appear in interactive testing (fix: load test, choose provisioned vs pay-per-token, warm replicas). (5) Observability — enable tracing + inference tables before launch to diagnose prod-only failures.

Section 4 — Assembling & Deploying Applications (22%)

Q59. MCQ

You must roll out RAG v2 to 10% of traffic, auto-revert if a quality scorer drops, and keep one stable reference for clients. Which combination is correct?

A. UC model aliases for the stable reference + endpoint traffic splitting for the 10% canary + automated scorer gate that repoints the alias on regression
B. Delete v1 and deploy v2 to 100%
C. Two separate workspaces with manual switching
D. A bigger model

✓ **Answer: A**

> Stable client reference = alias; partial rollout = traffic splitting across served entities; auto-revert = an automated evaluation gate that flips the alias/traffic back on regression. Deleting v1 removes rollback; separate workspaces aren't a canary mechanism; model size is irrelevant.

Q60. OPEN

Explain the precise migration risk created by the legacy inference table deprecation (post-Apr 30 2026) for a team that built monitoring on legacy payload tables, and the correct remediation.

✎ **Model answer:**

> After legacy inference tables are unsupported, pipelines and monitors built on the legacy payload schema/feature will stop receiving new data or break, silently degrading monitoring/evaluation. Remediation: migrate to AI Gateway-enabled inference tables (enable in the endpoint's AI Gateway config; default table <catalog>.<schema>.<endpoint>_payload), update unpacking/monitoring jobs to the new schema, and use Databricks' batch-migration notebook to move historical data. Validate that monitors and dashboards read the new tables before the cutoff.

Q61. MCQ

A served custom-model endpoint returns 429s and many requests are missing from the inference table. Which TWO facts explain the gaps and the throttling correctly?

A. Inference logs aren't guaranteed on errors (e.g., 4xx/5xx like 429) AND 429 means rate limiting — raise quota / add backoff or provisioned capacity
B. 429 means the model is too small AND logs are always complete
C. 429 is a success code AND missing logs mean Unity Catalog is off
D. Inference tables log everything AND 429 is a network DNS error

✓ **Answer: A**

> Inference tables may not record entries for endpoints returning errors (commonly 401/403/429/500 for custom models), explaining the missing rows; 429 specifically is throttling/rate limiting, addressed by higher quota, client backoff/retries, or provisioned capacity. The other pairs contain false statements.

Q62. OPEN

A deployed agent shows acceptable average latency but terrible p99, correlated with idle periods. Diagnose and give two mitigations with their trade-offs.

✎ **Model answer:**

> Diagnosis: scale-to-zero cold starts — after idle, the first request waits for compute to spin up, spiking p99. Mitigations: (1) keep a minimum of one warm replica (eliminates cold starts; trade-off: pay for idle capacity); (2) move to provisioned throughput for steady predictable latency (trade-off: reserved cost even at low volume); optionally (3) pre-warm before known traffic peaks (trade-off: scheduling complexity, still cold after unexpected idle). Choose based on latency-sensitivity vs cost tolerance.

Q63. MCQ

For per-team cost chargeback across external, provisioned, and pay-per-token endpoints, which data source is authoritative?

A. Inference (payload) tables
B. system.serving.endpoint_usage joined with served_entities (token/usage accounting), via AI Gateway usage tracking
C. The model signature
D. MLflow traces alone

✓ **Answer: B**

> Cost/usage accounting comes from the serving system/usage tables (endpoint_usage + served_entities) surfaced by AI Gateway usage tracking — they aggregate tokens/requests for chargeback across endpoint types. Payload tables capture content (not cost rollups); signatures and traces aren't billing sources.

Q64. MCQ

A team wants provider fallback: if the primary LLM degrades or errors, route to a backup automatically, with unified logging. Which layer provides this?

A. AI Gateway (routing/fallback + inference logging across providers)
B. Delta Live Tables
C. A Unity Catalog Volume
D. A notebook widget

✓ **Answer: A**

> AI Gateway centralizes routing/fallback to backup models on outage or quality degradation, with unified observability/inference logging. DLT, Volumes, and widgets don't route model traffic or provide fallback.

Q65. OPEN

Explain why registering to Unity Catalog (catalog.schema.model) materially improves multi-environment promotion compared to the legacy workspace registry.

✎ **Model answer:**

> UC's three-level namespace gives globally unique, governed identifiers with fine-grained, cross-workspace permissions and end-to-end lineage. You can structure dev/staging/prod as catalogs and promote a model across them with consistent governance, audited lineage from data → model → endpoint, and alias-based references. The legacy workspace registry is workspace-scoped with weaker, siloed governance, making cross-workspace promotion and unified lineage harder and less secure.

Q66. MCQ

A CI/CD pipeline should block promotion if faithfulness on a golden set falls below threshold. Where does this gate belong and what runs it?

A. A manual Slack vote
B. An automated mlflow.genai.evaluate step on a versioned eval dataset with faithfulness/correctness scorers, failing the build on regression
C. A check of model file size
D. Random sampling approval

✓ **Answer: B**

> The gate is an automated evaluation step (mlflow.genai.evaluate) on a versioned golden dataset with relevant scorers; if metrics regress below threshold, the pipeline fails and blocks promotion. Manual votes, file size, and random approval are not quality gates.

Q67. MCQ

An endpoint must enforce that no single user exceeds 100 requests/min, but a nightly ai_query batch over the same endpoint must run unthrottled. What's the correct understanding in 2026?

A. Set an AI Gateway per-user rate limit (applies to interactive serving requests); the ai_query batch is not rate-limited by the gateway, so it runs unthrottled while interactive users are capped
B. Rate limits apply equally to ai_query and interactive
C. You cannot rate-limit per user
D. ai_query bypasses Unity Catalog permissions

✓ **Answer: A**

> AI Gateway per-user rate limits govern interactive serving requests, capping users, while ai_query batch inference is usage-tracked but not gateway-rate-limited — so it runs unthrottled. B is false (asymmetry is the point), C is false (per-user limits exist), D is false (UC permissions still apply).

Q68. OPEN

Describe how MLflow traces for a deployed agent end up queryable for monitoring, and the size limitation to be aware of.

✎ **Model answer:**

> With AI Gateway inference tables enabled, the endpoint logs requests/responses and can store the agent's MLflow traces into a UC Delta table, which you query with SQL/notebooks for monitoring and feed into Lakehouse Monitoring. Limitation: the maximum logged request/response/trace size is 1 MiB; payloads exceeding it are logged as null with an error code (e.g., MAX_REQUEST/RESPONSE_SIZE_EXCEEDED), so very large traces/payloads may be dropped — design prompts/responses and trace verbosity with that cap in mind.

Q69. MCQ

To deploy an app composed of retrieval + prompt + LLM + custom business rules + parsing as ONE governed, rollback-able unit, you should:

A. Deploy 5 microservices wired by hand
B. Log it as a single MLflow model (PyFunc/chain) with signature + deps, register to UC, serve one endpoint, reference via alias
C. Email a zip to ops
D. Put each part in a separate notebook job

✓ **Answer: B**

> Packaging the whole app as one MLflow model gives atomic versioning, one signed endpoint, captured dependencies, UC governance, and alias-based rollback. Hand-wired microservices, zips, and scattered notebook jobs lack atomic versioning and governed rollback.

Q70. MCQ

A provisioned-throughput endpoint is overprovisioned (low utilization) and the bill is high. Volume is genuinely low and spiky. Best change?

A. Add more provisioned units
B. Switch to pay-per-token (or scale-to-zero) since low/spiky volume doesn't justify reserved capacity
C. Disable inference tables to save money
D. Increase max tokens

✓ **Answer: B**

> Low, spiky volume is the textbook case for pay-per-token (or scale-to-zero), eliminating reserved-capacity waste. Adding units worsens cost; disabling logging harms observability without fixing the capacity mismatch; max tokens is unrelated.

Q71. OPEN

Explain how you'd implement and verify a true zero-downtime model upgrade on a serving endpoint.

✎ **Model answer:**

> Register the new version to UC, add it as a second served entity on the SAME endpoint with a small traffic split (canary), and rely on readiness/health checks so traffic only routes once the new version is loaded and healthy. Monitor scorers/operational metrics on the canary slice; gradually shift traffic (e.g., 10%→50%→100%) if metrics hold, or repoint the alias/traffic back to the old version on regression. Because both versions coexist behind one endpoint and health checks gate routing, clients see no downtime. Verify with synthetic checks + live metric monitoring during the shift.

Q72. MCQ

Secrets for an external provider must be available to a deployed endpoint but never visible to developers or in logs. Correct mechanism?

- **A.** Databricks secret scopes referenced by the endpoint/config, with scoped access
- **B.** Put the key in the system prompt
- **C.** Commit to a private Git repo
- **D.** Pass it as a URL query parameter

✓ **Answer: A**

> Secret scopes inject credentials securely at runtime with access controls and keep them out of code, prompts, and logs. System prompts, Git commits, and URL parameters all expose secrets.

Q73. MCQ

Which statement about route-optimized vs standard serving endpoints and inference tables is currently accurate?

- **A.** Inference tables for route-optimized endpoints are in Public Preview; logs appear within ~1 hour and skip on certain errors
- **B.** Route-optimized endpoints cannot be served at all
- **C.** Inference tables are instant and always complete
- **D.** Route-optimized endpoints don't support Unity Catalog

✓ **Answer: A**

> Inference tables for route-optimized endpoints are in Public Preview; logs are typically available within about an hour and may be skipped for certain error responses. They are servable and UC-governed; logging is not instant or guaranteed-complete (B, C, D are false).

Q74. OPEN

A team enables AI Gateway guardrails for PII and prompt injection on a customer endpoint. Explain what guardrails add beyond Unity Catalog permissions, and one thing they do NOT do.

✎ **Model answer:**

> UC permissions control WHO can access which data/models/endpoints (access governance). Guardrails act at runtime on the CONTENT of prompts/responses/tool calls — detecting/blocking PII exposure, unsafe content, prompt injection, and data exfiltration — i.e., behavior governance the access layer can't see. What they don't do: guarantee factual correctness/eliminate hallucination by themselves (a hallucination guardrail can flag risk but isn't a correctness oracle), and they don't replace data-layer access control — you still need UC for who-can-read-what.

Q75. MCQ

After updating AI Gateway config (including a new rate limit) you don't see the limit take effect immediately. What's the expected behavior?

- **A.** Config never updates without redeploying the model
- **B.** Gateway config changes take ~20-40s to apply; rate-limit changes can take up to ~60s
- **C.** Changes require deleting the endpoint
- **D.** Rate limits apply instantly with zero delay

✓ **Answer: B**

> AI Gateway configuration changes apply in roughly 20-40 seconds, and rate-limit updates specifically can take up to about 60 seconds — so a brief delay is expected, not a bug. No redeploy or endpoint deletion is required.

Q76. MCQ

A Supervisor agent system must add a new Knowledge Assistant subagent at runtime via SDK, but the team already has 20 subagents. What happens?

- **A.** It silently replaces the oldest
- **B.** It fails — a supervisor system supports at most 20 agents; you must remove one or restructure
- **C.** It auto-creates a second supervisor
- **D.** It converts a subagent to a tool to make room

✓ **Answer: B**

> The documented hard limit is 20 agents per supervisor system; exceeding it isn't allowed. You must remove/consolidate subagents or restructure (e.g., hierarchical supervision). It won't silently replace, auto-create, or auto-convert.

Q77. OPEN

Explain end-to-end how a RAG app's Unity Catalog lineage helps during a production incident where answers suddenly cite wrong documents.

✎ **Model answer:**

> UC lineage links source tables → chunk/Delta tables → vector index → registered model → serving endpoint. During the incident you trace backward: confirm which source table/version feeds the index, whether a recent ingestion changed it, whether the index synced to the wrong/old data, and which model version/prompt alias is live. Lineage pinpoints where the bad data entered (e.g., a wrong upstream load) and what downstream assets are affected, enabling targeted rollback (repoint alias, re-sync index, revert source load) rather than guesswork.

Q78. MCQ

The cheapest correct way to serve a 5M-row nightly classification job, given no latency SLA, is:

- **A.** A hot real-time endpoint at max concurrency 24/7
- **B.** Batch inference via ai_query (or scheduled batch scoring), no always-on real-time endpoint
- **C.** Manual per-row API calls
- **D.** A streaming UI

✓ **Answer: B**

> No latency SLA + large offline volume = batch inference (ai_query/scheduled batch), which avoids paying for an always-on endpoint. A wastes money; manual calls don't scale; streaming UIs are for interactive use.

Q79. MCQ

Which is a correct reason a logged model fails to serve despite working locally, and the fix?

- **A.** Missing/incorrect captured dependencies — re-log with accurate pip/conda requirements and an input example, then test-load in a clean env
- **B.** The model is too accurate
- **C.** Unity Catalog is incompatible with serving
- **D.** Signatures prevent serving

✓ **Answer: A**

> Local success but serving failure typically means the model's recorded environment is missing a dependency the local env had. Fix by capturing correct requirements (and an input example) and validating in a clean environment. UC and signatures enable, not block, serving.

Q80. OPEN

Why is enabling inference tables BEFORE launch (not after an incident) a deployment best practice for GenAI apps?

✎ Model answer:

> Inference tables only capture traffic after they're enabled; if you enable them post-incident you have no historical record of the requests/responses (and traces) that caused the problem, crippling root-cause analysis, evaluation, and drift baselining. Enabling pre-launch ensures you accumulate the production data needed for monitoring, quality scoring, drift detection, and building training/fine-tune corpora from day one — and establishes a baseline to detect regressions against.

Section 5 — Governance, Security & Privacy (8%)

Q81. MCQ

A RAG chatbot must ensure a sales rep retrieves only their own region's accounts, enforced even if the rep crafts a clever prompt. The correct enforcement is:

A. A system-prompt rule 'only show your region'
B. Unity Catalog row filtering on the source/view, evaluated under the rep's identity, so unauthorized rows are never retrieved
C. Asking the LLM to self-censor
D. Post-processing the answer to remove other regions

✓ **Answer: B**

> Row filtering at the data layer under the caller's identity guarantees out-of-region rows are never even retrieved — injection-proof. Prompt rules and LLM self-censorship are bypassable; post-processing is fragile and can leak before filtering. Enforce at the source.

Q82. OPEN

Distinguish the governance roles of (a) column masking, (b) row filtering, and (c) AI Gateway guardrails in a single PII-sensitive RAG deployment, with one example each.

✎ **Model answer:**

> (a) Column masking transforms sensitive fields at the data layer so even authorized retrieval never exposes raw values — e.g., SSN shown as ***-**-1234 in the chunk source. (b) Row filtering restricts which records a principal can access — e.g., a clinician sees only their patients' notes. (c) AI Gateway guardrails act at runtime on prompt/response CONTENT — e.g., blocking a response that would leak a credit-card number or detecting a prompt-injection attempt. Masking/row-filtering prevent unauthorized DATA from reaching the model; guardrails prevent unsafe CONTENT from reaching the user. Defense in depth uses all three.

Q83. MCQ

For GDPR right-to-erasure in a deployed RAG system, which is BOTH necessary and sufficient to truly remove a person's data from answers?

A. Add a prompt instruction to ignore that person
B. Delete the person's rows in the source Delta table (with stable IDs) so the synced vector index drops the corresponding vectors, and purge them from inference/log tables per retention policy
C. Lower the temperature
D. Retrain the base model

✓ **Answer: B**

> True erasure requires deleting the underlying rows (using stable IDs) so the synced index removes those vectors, AND purging any retained copies in inference/log tables per policy — otherwise data persists in logs or the index. Prompt instructions don't delete data; temperature is irrelevant; retraining the base model is neither necessary nor sufficient for record-level erasure.

Q84. MCQ

A team wants to commercially deploy a fine-tuned open model. Which governance check is decisive and most often overlooked?

A. The model's benchmark score
B. Whether the base model's LICENSE permits commercial use AND derivative/fine-tuned redistribution under your intended terms
C. The model's parameter count
D. The color of the model card

✓ **Answer: B**

> Licensing governs legal commercial use and whether fine-tuned derivatives can be used/redistributed as intended — frequently overlooked and decisive. Benchmarks, size, and cosmetics don't determine legal rights.

Q85. OPEN

Explain why logging full prompts/responses to inference tables can itself create a compliance violation, and how to keep observability without the violation.

✎ **Model answer:**

> Prompts/responses can contain PII, secrets, or regulated data; persisting them verbatim in inference tables creates a new copy of sensitive data subject to access, retention, and residency rules — a potential violation if uncontrolled. Mitigate by masking/redacting PII before or at logging, restricting access to the inference tables via Unity Catalog permissions, applying retention/expiration policies and column masking, and ensuring region/residency compliance. You keep observability (metrics, traces, debugging) while limiting exposure of raw sensitive content.

Q86. MCQ

Which is the strongest argument that access control belongs at the retrieval layer rather than the generation layer?

A. Generation is slower
B. If unauthorized data is retrieved into the context, it can leak via paraphrase/summarization even if you instruct the model not to reveal it; never retrieving it is the only robust guarantee
C. Retrieval is cheaper
D. The model prefers fewer tokens

✓ **Answer: B**

> Once unauthorized content is in context, the model can leak it indirectly (paraphrase, summary, inference) regardless of instructions; the only robust control is to never retrieve it (data-layer filtering). Speed, cost, and token preferences are not the governance argument.

Q87. MCQ

An auditor asks 'which source documents and which model/prompt version produced this March answer?' Which capabilities together satisfy this?

A. Inference tables (request/response + trace, identity) + Unity Catalog lineage + prompt/model versioning
B. Higher temperature
C. A bigger embedding model
D. Disabling logging for privacy

✓ **Answer: A**

> Reconstructing a past answer's provenance needs the logged request/response and trace (inference tables), lineage from source data to model/endpoint (UC), and the exact prompt/model versions (registry/aliases). Temperature/embedding size don't provide provenance; disabling logging destroys auditability.

Q88. OPEN

A workspace must serve EU customers with data residency constraints while using a third-party model. Explain the governance levers and the residual risk to flag.

✎ **Model answer:**

> Levers: deploy in an EU region/workspace; govern data and assets with Unity Catalog (permissions, lineage, masking); route the external model through an AI Gateway external endpoint so requests, logging, and policy are centralized and auditable; apply guardrails to prevent exfiltration of regulated data; set inference-table location/retention in-region. Residual risk to flag: a third-party model provider may process or transit data outside the EU depending on where the provider runs — you must verify the provider's processing region/contractual terms, since region-selecting your workspace does not by itself guarantee the external provider keeps data in-region.

Section 6 — Evaluation & Monitoring (12%)

Q89. MCQ

A RAG system scores high Faithfulness, high Context Recall, but LOW Answer Relevancy. What is the most likely diagnosis?

A. Retrieval is broken
B. The generator is grounded and the right context is present, but it's answering a different/over-broad question — a generation/prompt-framing problem (or query misinterpretation)
C. Embeddings are too small
D. The vector index is empty

✓ **Answer: B**

> High faithfulness + high context recall means grounding and retrieval coverage are good; low answer relevancy means the output doesn't address the user's actual question — typically prompt framing/task interpretation or query understanding. It's not a retrieval failure (recall is high) or an empty index.

Q90. OPEN

You can run an LLM-as-judge for faithfulness cheaply at scale, but it disagrees with SMEs ~20% of the time. Describe a defensible production evaluation strategy that uses both.

✎ **Model answer:**

> Use the LLM judge for broad, continuous coverage on all/most traffic to flag likely issues and trends, while calibrating it against a SME-labeled golden set: measure judge-vs-human agreement, refine the judge rubric/prompt to close the gap, and route a sampled subset (especially low-confidence or high-stakes cases) to SME review via the Review App. Use SME feedback to expand the golden dataset and recalibrate periodically. Report metrics with the known judge error rate, and never treat the judge as ground truth for high-stakes decisions without human spot-checks.

Q91. MCQ

A classifier must minimize FALSE NEGATIVES (missing fraud) even at the cost of more false positives. Optimizing which metric is correct, and which is a trap?

A. Maximize Recall; trap = optimizing Accuracy on an imbalanced set (a 99%-legit dataset scores 99% by predicting 'legit' always)
B. Maximize Precision; trap = Recall
C. Maximize BLEU; trap = ROUGE
D. Maximize Perplexity; trap = F1

✓ **Answer: A**

> Minimizing false negatives = maximizing Recall. The classic trap is Accuracy on an imbalanced dataset, where a trivial majority-class predictor looks excellent while catching zero fraud. BLEU/ROUGE/Perplexity are generation/LM metrics, irrelevant to fraud classification.

Q92. MCQ

You must enforce 'every answer cites at least one valid source ID present in the retrieved context.' Which evaluation construct is correct?

A. BLEU against a reference
B. A custom code-based scorer that parses citations and verifies each ID exists in the retrieved context, run via mlflow.genai.evaluate
C. Perplexity
D. Context Recall alone

✓ **Answer: B**

> This is a deterministic structural/grounding rule best checked by a custom code-based scorer that validates cited IDs against the retrieved context. BLEU/perplexity don't check citation validity; context recall measures retrieval coverage, not citation correctness.

Q93. OPEN

Explain the CLEARS framework and why monitoring only Correctness in production is insufficient for an agent.

✎ **Model answer:**

> CLEARS evaluates agents across Correctness, Latency, Execution (did tool use/steps run properly), Adherence (to instructions/guidelines), Relevance, and Safety. Monitoring only Correctness misses critical production failure modes: latency regressions (UX/SLA), execution failures (tools erroring or wrong tool order), drift from guidelines (adherence), off-topic answers (relevance), and unsafe/PII-leaking outputs (safety). An agent can be 'correct' on average while violating SLAs, leaking data, or mis-executing tools — so multi-dimensional monitoring is required.

Q94. MCQ

A served RAG app's payload table must be monitored for quality drift over time. Which Lakehouse Monitoring profile is correct, and what preprocessing is needed first?

A. Snapshot monitor; no preprocessing
B. InferenceLog monitor; first unpack the JSON payloads (per endpoint schema) and optionally join ground-truth labels, then create/refresh the monitor
C. TimeSeries monitor on the raw blob; no schema needed
D. No monitor is possible on inference tables

✓ **Answer: B**

> Serving request logs over time use the InferenceLog profile. First unpack the JSON request/response payloads according to the endpoint schema (and optionally join labels for quality metrics), then create a monitor over the resulting Delta table and refresh it. Snapshot is current-state only; raw-blob TimeSeries won't yield model-quality metrics; monitoring inference tables is explicitly supported.

Q95. MCQ

Which combination distinguishes DATA drift from CONCEPT drift in a deployed GenAI app, and the correct response to each?

A. Data drift = input distribution shifts (e.g., new question types) → refresh corpus/retrieval; concept drift = the input→correct-output relationship shifts (e.g., policy changed) → update ground truth/prompts/model and re-evaluate
B. They are the same thing
C. Both are fixed by raising temperature
D. Both require deleting the endpoint

✓ **Answer: A**

> Data drift is a change in inputs (distribution), addressed by refreshing the knowledge corpus/retrieval and re-baselining; concept drift is a change in what the correct answer should be (the relationship), addressed by updating ground truth/prompts/model and re-evaluating. They're distinct; temperature and endpoint deletion don't address either.

Q96. OPEN

Why should the SAME scorers be used in development and production monitoring, and what failure does using different metrics introduce?

✎ **Model answer:**

> Reusing identical scorers ensures the quality you measured and optimized in development is the quality you track in production — an apples-to-apples signal that lets you detect real regressions and attribute them to changes. Different dev vs prod metrics introduce a measurement gap: production 'looks fine' on a different metric while the property you tuned for silently degrades, or vice versa, making regressions undetectable or misattributed. MLflow 3 reuses the same built-in/custom scorers across dev and prod precisely to avoid this.

Q97. MCQ

An LLM-as-judge for 'helpfulness' rates verbose answers higher than concise correct ones. This is an example of, and is mitigated by:

A. Verbosity/length bias; mitigate with a clearer rubric (penalize unnecessary length), reference answers, and calibration against human labels
B. Data drift; mitigate by retraining
C. A retrieval bug; mitigate by raising k
D. A latency issue; mitigate by scaling

✓ **Answer: A**

> Preferring longer outputs is a known LLM-judge verbosity/length bias. Mitigate by tightening the rubric (explicitly value concise correctness, penalize padding), supplying reference answers, and validating/calibrating the judge against human labels. It isn't drift, retrieval, or latency.

Q98. OPEN

Design an evaluation dataset strategy for a RAG app: what should it contain, how does it relate to traces and the Review App, and why version it in Unity Catalog?

✎ **Model answer:**

> It should contain representative and edge-case questions with expected facts/answers (and ideally the expected supporting context), including hard, ambiguous, and adversarial cases. Build it by converting real production TRACES into evaluation records and by collecting SME judgments through the Review App (human feedback attached to traces). Version it in Unity Catalog so evaluations are reproducible and comparable across model/prompt versions, with lineage and governance — letting you re-run the same dataset on every candidate, track quality over time, and prove which change caused a regression.

Q99. MCQ

For summarization quality, relying on ROUGE alone is risky because:

A. ROUGE measures latency
B. ROUGE rewards n-gram overlap with a reference but can score a fluent, factually-wrong summary highly; pair it with a faithfulness/factuality check
C. ROUGE guarantees factual accuracy
D. ROUGE is a retrieval metric

✓ **Answer: B**

> ROUGE captures lexical overlap with a reference, not factual correctness, so a fluent but unfaithful summary can score well. Complement it with a faithfulness/factuality judge. It isn't a latency or retrieval metric and doesn't guarantee accuracy.

Q100. MCQ

Production dashboards show stable quality scorers but rising user thumbs-down and rising p95 latency. What is the most defensible conclusion?

A. Everything is fine because scorers are stable
B. Quality content may be acceptable while EXPERIENCE is degrading (latency) and/or scorers miss what users care about; investigate latency and expand metrics/feedback signals
C. Lower temperature to fix latency
D. Delete the dashboards

✓ **Answer: B**

> Stable quality scorers plus rising dissatisfaction and latency suggests operational degradation (latency) and/or a coverage gap in your metrics versus real user needs. Investigate the latency regression and broaden monitoring (additional scorers, feedback analysis). Ignoring it (A), temperature (C, unrelated to latency), and deleting dashboards (D) are wrong.